

## [L0-L1 核心本质与设计逻辑]

**L0 一句话本质：**在非易失性存储与易失性内存的物理界限下，通过并发控制与预写日志保证 ACID 强一致约束，并借由查询优化器与存储引擎最大化 I/O 与 CPU 吞吐的关系型计算系统。

**L1 四句话逻辑：**

- 并发工作流调度 (M1)** 通过进程/线程池模型降低上下文切换开销，配合逻辑锁（Latch/Mutex）隔离多线程内存竞争。
- 查询处理器优化 (M2)** 建立逻辑计划并利用 CBO 代价模型搜索生成最优物理执行树，通过 Volcano 迭代器或向量化流式计算。
- 缓冲池 Clock 淘汰与槽页 (M3)** 将物理磁盘页组织为变长槽页并在缓存中 Pin/Unpin 锁页，平滑屏蔽随机 I/O 延迟。
- SS2PL 锁并发与 ARIES 恢复 (M4 & M5)** 构成事务的一致性防线，通过锁等待图打破死锁，利用 WAL 日志三阶段分析回滚恢复崩溃状态。

## [L2 存储引擎数据流演进拓片]

• [SQL 客户端连接] → [工作线程分派] → [语法解析与 CBO 优化] → [查询物理执行计划] → [SS2PL 锁管理器校验] → [缓冲池 Pin 锁页] → [物理页读写] → [WAL 预写量 fsync] → [物理脏页 Clock 淘汰刷盘]

## [M1.1] 每连接单进程模型 (Process-per-Connection Model)

- 早期数据库首选：**PostgreSQL 早期及 Oracle 采用多进程架构。操作系统为每个客户端连接派生一个独立的 OS 进程。
- 物理瓶颈：**进程间通信（IPC）必须通过共享内存（Shared Memory）；进程创建和上下文切换开销极其昂贵，在连接数超千时会导致宿主机 CPU 耗尽。

## [M1.2] 每连接单线程模型 (Thread-per-Connection Model)

- 多线程共享开销小：**MySQL 及 SQL Server 采用每连接单线程模型。所有线程共享同一个进程的虚拟地址空间。
- 堆栈碎片瓶颈：**每个线程依然拥有独立的局部堆栈。当连接数过万时，线程调度开销和堆栈内存碎片依然会导致系统性能退化。

## [M1.3] 工作线程池异步调度 (Worker Pool Scheduling)

- 工作线程池复用：**现代高效数据库（如 MySQL Thread Pool）将连接和执行分离。构建固定大小的 Worker 线程池。
- 异步多路复用：**前端使用 epoll 等异步多路复用机制接收 Socket 请求，将其封装为任务放入队列，由后台 Worker 线程竞争执行，实现了超低开销的万级高并发连接。

## [M1.4] Latch 与 Mutex 轻量锁 (Latch vs Mutex)

- 语义区别：**在数据库中，Lock 用于保护数据库逻辑事务一致性（对用户可见）；而 Latch (包括 Mutex 和 RWLock) 用于保护进程内的共享内存数据结构（如缓冲池页表、B-Tree 节点指针）。
- 短平快设计：**Latch 极其轻量，通常由 CPU CAS 指令实现自旋锁，只在毫秒级内持有，用于保证内部多线程并发下的数据安全。

## [M2.1] SQL 词法解析与重写 (SQL Parsing &amp; Rewriting)

- 词法分析树：**Parser 接收 SQL 文本，通过词法和语法分析将其翻译为 Abstract Syntax Tree (AST，抽象语法树)。
- 语义去重扁平化：**Query Rewriter (重写器) 接收 AST，进行权限校验、视图（View）扁平化展开以及常量折叠重写，将其转换为逻辑查询计划（Logical Query Plan）。

## [M2.2] 基于代价优化器 CBO (Cost-Based Optimizer)

- 代价搜索空间：**CBO 接收逻辑计划，评估每种物理算子执行路径的总开销。
- 物理代价公式：**优化器基于收集到的物理表元数据（如行数、直方图选择度、索引高度），使用公式  $Cost = Page\_Reads \times W_{IO} + Tuples\_Scanned \times W_{CPU}$  计算每个路径的总代价，并选择代价值最小的计划作为物理计划（Physical Plan）。

## [M2.3] System R 优化路径决策模型 (System R Dynamic Programming Join Plan)

- Join 路径爆破：**多表 Join 的物理排列组合呈指数级增长。System R 优化器利用动态规划（Dynamic Programming）算法自底向上搜索 Join 树。
- 左深树截断：**通过限制只生成左深树（Left-Deep Tree，即 Join 的右子节点必须是原始基表，方便流水线式处理），大幅裁剪搜索空间，选出最优的多表 Join 连接顺序。

## [M2.4] Volcano 火山迭代器执行模型 (Volcano Iterator Model)

- 算子调用约定：**最经典且广泛使用的执行引擎模型。每个物理算子（如 SeqScan, HashJoin）均继承统一接口，实现 `open()`, `next()`, `close()` 三个方法。
- 拉取流水线：**父算子循环调用子算子的 `next()` 获取一行记录（Tuple），直到子算子返回 EOF。数据自下而上流式流动，内存占用极小。

## [M2.5] 向量化执行与 JIT 编译 (Vectorized Execution vs JIT)

- 传统火山模型痛点：**对于 OLAP 大批量扫描，火山模型中每处理一行都会产生一次 C++ 虚函数调用，CPU 指令分支预测失败率极高。
- 两大进化路线：**
  - 向量化执行：**每个 `next()` 调用不返回一行，而是返回一个 Tuple 数组（如 1024 行），利用 CPU 缓存与 SIMD 寄存器加速。
  - JIT 即时编译：**利用 LLVM 运行时将整条查询编译为单条机器码，消除虚函数调用与算子间跳转开销。

## [壹] 数据库并发与崩溃恢复折衷与异常防范矩阵 (Database Concurrency &amp; Recovery Trade-offs)

**细粒度行锁能实现最高并发且毫无副作用。数万个行锁会耗尽管理器的内存哈希表，频繁的锁升级（Lock Escalation）会导致大面积的行锁变为表锁，从而引发事务级串行死锁。** → 限制锁分配总数。当特定事务持有的行锁数超过配置阈值时，锁管理器主动将其收缩合并并升级，并在高频写冲突业务中使用乐观锁 OCC 或分区 MVCC 规避。

**崩溃恢复时直接回滚未提交的事务即可。直接逆向回滚会被坏历史戳覆盖物理脏页状态，导致数据文件发生串同步残缺。ARIES 算法要求“先重做历史（Repeating History）”，即使未提交事务也必须先重做。** → 严格执行 ARIES 三阶段恢复：先分析日志，再重做（Redo）包含未提交事务在内的所有历史页状态到崩溃前夕，最后通过 Undo 逆向执行单向回滚以确保物理安全性。

**利用 LRU 链表可以完美实现缓冲池页淘汰；在高并发多线程读写下，频繁调整 LRU 双向链表头尾指针需要争抢全局互斥锁（Mutex Latch），这会导缓冲池淘汰机制瞬间沦为系统性能瓶颈。** → 启用 Clock（时钟）替换算法。将缓冲页以环形数组组织，通过循环指针和使用位（usage\_count）自减判断，避免了频繁锁链表的开销，实现无锁/轻量级锁淘汰。

## [M4.1] 严格两阶段锁协议 (SS2PL) (Strict Two-Phase Locking)

- **串行化保证**: SS2PL 要求事务在加锁增长期 (Growing Phase) 可以申请锁, 但在释放缩编期 (Shrinking Phase) 只能在 **事务提交或回滚时一次性释放所有排他锁 (X Lock)**。
- **崩溃级联回滚**: 这种机制保证了事务持有的未提交修改数据绝不会被其他事务看到, 消除了脏读, 防止系统发生灾难性的级联回滚。

## [M4.2] 锁管理器物理哈希表结构 (Lock Manager Layout)

- **内锁链表结构**: 锁管理器维护一个全局的哈希表。
- **双链等待**: 哈希的 Key 是数据库资源 ID (如页 ID 或行 ID), Value 是锁头结构 (Lock Head)。锁头内部链接两个双向链表: Granted List (已授予锁的事务链表) 和 Wait List (正在等待该资源的事务排队链表)。通过此结构进行毫秒级冲突判断。

## [M4.3] 锁升级与等待图死锁检测 (Lock Escalation &amp; Deadlock)

- **锁膨胀防御**: 当大量并发读写小资源 (行锁) 导致锁表耗尽内存时, 锁管理器启动锁升级 (Lock Escalation), 将一定数量的细粒度锁合并为高层级的粗粒度锁 (如行锁变表锁)。
- **死锁破连环**: 通过后台线程定期构建锁等待有向图 (Wait-For-Graph), 检测图中是否存在有向环路。若检测到环路, 选择代价最小的事务 (Victim) 将其强行 Abort 并回滚, 释放锁以打破死锁死结。

## [M4.4] MVCC 版本链隔离快照 (MVCC Version Chain)

- **读写完全不互斥**: 多版本并发控制下, 写事务并不覆盖原记录, 而是在 Undo Log 中构建一条物理版本链, 记录每次修改的增量。
- **Read View 读视图**: 只读事务启动时获取当前活动事务的 Read View 视图。读取数据时, 沿着版本链向后寻找, 只读取对于当前 Read View 可见的、已提交的历史版本, 实现了读不加锁、读写不冲突。

## [M4.5] 乐观并发 OCC 冲突校验 (OCC Validation)

- **无锁并发设计**: 乐观并发控制假设写冲突极少发生。事务执行阶段完全不加锁, 将所有修改写入私有工作区 (Private Workspace)。
- **三阶段校验**: 在 Validation (校验) 阶段, 系统检查该事务读取的记录是否在其执行期间被其他已提交事务修改。如果存在冲突, 则回滚该事务工作区并重试; 若无冲突, 在 Write (写) 阶段将工作区物理覆盖至主库。

## [M5.1] WAL 预写日志协议 (WAL Protocol)

- **数据不丢铁律**: 脏页物理写回磁盘前, **描述该页修改的日志记录 (WAL Log) 必须先刷入非易失性存储磁盘**。
- **事务持久化**: 事务提交时, 必须等待该事务涉及的所有 WAL 日志执行完 fsync 物理落盘, 才向客户端返回提交成功, 保证了崩溃后的绝对可回溯。

## [M5.2] ARIES 恢复分析阶段逻辑 (ARIES Analysis Phase)

- **定位数据起点**: 系统崩溃重启后, 首先读取最后一个 Checkpoint 记录, 进入 Analysis (分析) 阶段。
- **重构状态表**: 从该检查点向后扫描日志, 重建崩溃前夕的活跃事务表 (Transaction Table, TT) 和脏页表 (Dirty Page Table, DPT), 找出最旧的未刷盘脏页对应的 LSN, 作为下一步 Redo 的起点。

## [M5.3] ARIES 恢复重做阶段逻辑 (ARIES Redo Phase)

- **重放历史历史 (Repeating History)**: 从分析阶段确定的脏页起点 LSN 开始, 正向扫描日志, 重新执行所有 WAL 日志记录的修改操作 (无论该事务最终是否提交), 将数据库物理状态完全恢复到崩溃前的瞬间。

## [M5.4] ARIES 恢复撤销阶段逻辑 (ARIES Undo Phase)

- **物理状态大扫除**: 重做完成后, 事务状态表完全还原到崩溃前。此时进入 Undo (撤销) 阶段。
- **逆序回滚**: 系统从最新日志处逆序反向扫描, 依次撤销崩溃前夕处于未提交状态的所有事务。对于每个回滚动作, 写入一条补偿日志记录 (Compensation Log Record, CLR), 防止撤销过程中再次发生崩溃时重复回滚。

## [M5.5] 模糊检查点机制原理 (Fuzzy Checkpointing)

- **不停机写元数据**: 如果检查点操作需要将内存所有脏页完全刷盘, 会导致系统在刷盘期间完全停摆 (Stall)。
- **增量元数据落盘**: Fuzzy Checkpoint 只记录脏页表 (DPT) 和活跃事务表 (TT) 的当前快照状态, 并写一条 Checkpoint 日志, 完全不强制刷脏数据页, 实现了检查点操作与正常读写事务的完美并行。

## [M6.1] 共享内存架构物理限制 (Shared-Memory Architecture)

- **硬件总线天花板**: 在 Shared-Memory 架构下, 所有 CPU 共享物理主内存及外部磁盘。
- **一致性瓶颈**: 随着 CPU 核心数增加, 内存总线争抢和 CPU 高速缓存一致性 (Cache Coherency) 协议维护开销会呈指数级上升, 硬件扩展性通常在数十核达到物理天花板。

## [M6.2] 共享磁盘架构锁协调代价 (Shared-Disk Architecture)

- **SAN 网络互连**: 每个节点拥有独立的内存, 但通过专用的高速存储局域网 (SAN/NAS) 共享底层的全部物理磁盘。
- **分布式锁开槽**: 由于不同节点的内存独立, 为了防止不同节点并发读写同一个磁盘块, 需要在节点之间通过网络建立极其复杂的分布式锁管理器 (DLM) 协议, 锁协调开销较大。

## [M6.3] 无共享架构水平扩展原理 (Shared-Nothing Architecture)

- **物理完全隔离**: 每一个物理节点 (Broker/Node) 都拥有独立且完全隔离的 CPU、内存和磁盘。节点之间只能通过高速以太网发送消息进行通信。
- **水平无限扩展**: 数据通过分区 (Sharding) 散列到不同节点上, 这是现代超大规模分布式数据库 (如 CockroachDB、ClickHouse) 实现近乎无限水平扩展的黄金架构模式。

## [M6.4] 分布式二阶段提交协议 (Two-Phase Commit)

- **分布式一致性防线**:
  1. **准备阶段 (Prepare)**: 协调者向所有参与节点广播 Prepare 询问, 节点本地执行并锁住资源, 返回 Ready 或 Abort。
  2. **提交阶段 (Commit)**: 只有所有节点都返回 Ready, 协调者才广播 Commit 指令; 只要有一个节点返回 Abort 或超时, 协调者广播 Rollback 回滚, 确保分布式状态原子闭环。

## [M6.5] 分布式查询数据重分布与 Join (Data Redistribution &amp; Distributed Join)

- **海量计算网卡传输**: 分布式多表 Join 时, 如果数据存放在不同节点, 必须进行网络移动。
  1. **三大 Join 选型**:
    1. **Colocated Join**: Join 的键已经是相同的分区方式, 直接本地 Join。
    2. **Broadcast Join**: 把小表广播复制发送给大表的所有节点。
    3. **Shuffle Join**: 大小表按 Join 键重新哈希, 通过网络进行重分布传输后再本地 Join, 折衷网卡流量与 CPU 负荷。

## [贰] Zone T 数据库性能与诊断实验室 (Database Performance &amp; Diagnostic Labs)

## T1 关系型数据库典型参数调优对照表

T2 经典数据库运维监控视图字典

- 1. 诊断当前慢查询与锁阻塞活动

```
sql
-- PostgreSQL: 查看所有非空闲状态的进程及其实时查询文本
SELECT pid, username, query, state, age(clock_timestamp(), query_start) AS duration
FROM pg_stat_activity
WHERE state != 'idle' ORDER BY duration DESC;
```

- 2. 分析全局锁竞争与等待队列

```
sql
-- PostgreSQL: 查询哪些进程持有了锁, 哪些进程正在 Wait List 中排队挂起
SELECT a.pid AS blocked_pid, a.query AS blocked_query,
       l.locktype, l.mode, l.granted,
       d.pid AS blocking_pid, d.query AS blocking_query
FROM pg_catalog.pg_locks l
JOIN pg_catalog.pg_stat_activity a ON l.pid = a.pid
JOIN pg_catalog.pg_locks blocked_by ON l.locktype = blocked_by.locktype
AND l.database IS NOT DISTINCT FROM blocked_by.database
AND l.relation IS NOT DISTINCT FROM blocked_by.relation
AND l.page IS NOT DISTINCT FROM blocked_by.page
AND l.tuple IS NOT DISTINCT FROM blocked_by.tuple
AND l.virtualxid IS NOT DISTINCT FROM blocked_by.virtualxid
AND l.transactionid IS NOT DISTINCT FROM blocked_by.transactionid
AND l.classid IS NOT DISTINCT FROM blocked_by.classid
AND l.objid IS NOT DISTINCT FROM blocked_by.objid
AND l.objsubid IS NOT DISTINCT FROM blocked_by.objsubid
AND l.pid != blocked_by.pid
JOIN pg_catalog.pg_stat_activity d ON blocked_by.pid = d.pid
WHERE NOT l.granted;
```

- 3. InnoDB 引擎内部监控诊断

```
sql
-- MySQL: 查看死锁日志、事物链表、Latch 等锁状态以及缓冲池淘汰详情
SHOW ENGINE INNODB STATUS;
```